

Trellis

让 AI 在项目规则中持续协作

把流程、任务、规范与经验沉淀在仓库里，让每次协作都能接得上。

一句话定义

Trellis 是放在项目仓库里的 **AI 开发协作框架**，用统一流程连接需求规划、项目规范、任务执行、质量检查和经验沉淀。

价值

让 AI **先读规则、按任务工作、留下可复核记录**。

边界

它不是大模型，不替代 Git，也不保证代码天然正确。



为什么 AI 开发需要一条“可接续”的线

问题不在于 AI 能不能回答，而在于工作能否持续、可追踪地推进。

常见断点

- 新会话开始，又要重新解释目标和背景；
- AI 没有读到项目约定，做法逐渐漂移；
- 需求、设计、检查结果散在对话里，难以复核；
- 多人或多个角色协作时，进度与责任不清晰。

Trellis 的补法

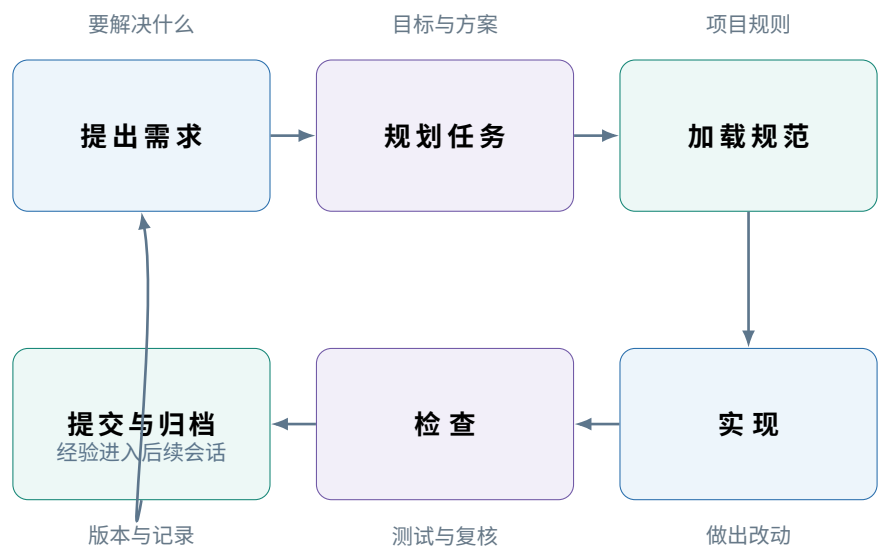
- 用 workflows 提示当前阶段和下一步；
- 用任务文件保存目标、设计、计划和结果；
- 用项目规范约束实现前必须阅读的规则；
- 用工作区与记忆保存进展和可复用经验。



关键变化 从“每次给 AI 一段临时提示”，变成“让 AI 在项目已有的上下文中完成一项可复核的工作”。

一项工作怎样在 Trellis 中流动

先规划，再实现；先读规范，再动手；完成后留下记录。



环节	它解决什么
规划任务	先把目标、范围、验收标准和执行思路写清楚，避免直接开写后反复返工。
加载规范	AI 在实现前读取项目已有的约定，减少“每个任务一套做法”。
检查与归档	测试、评审、差异检查仍然执行；结果与经验被保留给下一次使用。

分工不变，只是衔接更清楚 Git 管版本；AI 模型负责理解与生成；测试和评审判断质量；Trellis 补充流程与上下文。

四类内容，把协作放回项目里

组成	通俗解释
Workflow	规定现在处于哪一步、下一步该做什么，让一次工作有明确节奏。
Tasks	保存一项工作的目标、设计、执行计划和结果，让过程可追踪、可复核。
Specs	保存这个项目自己的工程规范与边界，让 AI 不必靠猜测写代码。
Workspace / Memory	保存会话进展和经验，让下一次协作可以从已知状态继续。

一个不含业务细节的例子：为产品增加一个功能

1. 先说清楚

写下功能目标、范围和验收条件；必要时先形成设计与实施计划。
2. 再读取规则

AI 加载相关项目规范，了解既有代码风格、测试要求和边界。
3. 实现并检查

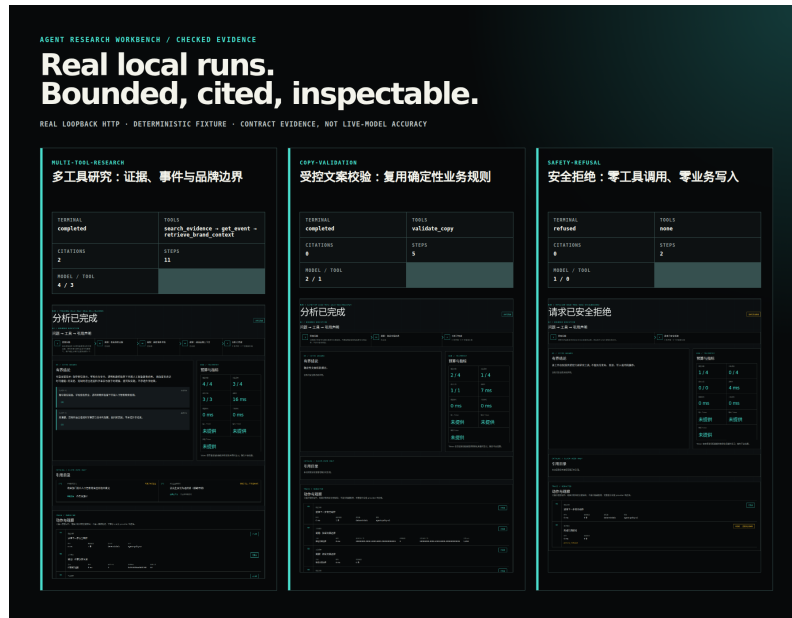
按计划改动，运行测试与质量检查；问题回到任务记录中处理。
4. 提交并留痕

用 Git 保存代码版本；任务结果与会话经验成为下一次工作的起点。

有了这四类内容 不靠某个人记住上下文，也不把关键决策只留在一段难以回看的聊天记录中。

真实项目运行记录：每一步都能回看

以下是项目中的真实本地运行记录，并非 Trellis 自身的界面：运行使用脱敏 deterministic fixture，展示可复现的本地 loopback API/UI 执行、工具边界与可检查记录。



真实项目本地运行总览：完成、受控校验与安全拒绝都留下可查看的结果。



多工具详情：问题、受控工具、引用结论与过程观测在同一条运行记录中对应。

图中使用脱敏确定性夹具，证明本地执行链和可检查边界；不代表线上模型能力或生产运行情况。

适合谁用，以及该怎样理解它的价值

适合的工作

- 有多个步骤、需要设计和验收的功能开发；
- 要跨会话推进、容易丢失背景的长期事项；
- 有团队规范，希望 AI 也能遵守的项目；
- 需要多人或多个角色协作、追踪交接的任务。

不该期待的事

- 它不会替你选择业务方向；
- 它不取代 Git 的版本控制；
- 它不替代测试、评审与人的判断；
- 它不能承诺每一次生成都没有错误。

**Trellis 的价值不是替代人或工具，
而是让 AI 辅助开发有流程、有上下文、有记录。**

小结：从一项具体任务开始，把“先读规则、按任务工作、留下记录”变成团队习惯。